

PRÁCTICA EVOLUTIVA

HelpDesk Lite

Sistema de gestión de tickets de soporte técnico

HTML5 · CSS3 · JavaScript Vanilla · DOM · localStorage · Git · GitFlow

Materia: Aplicaciones de Internet

Modalidad: práctica individual o por parejas

1. Propósito de la práctica

Desarrollar una aplicación web frontend que permita registrar y administrar tickets de soporte técnico. La práctica busca integrar lo aprendido previamente sobre HTML semántico, CSS responsive, JavaScript, DOM, eventos, arrays, objetos, filtros y localStorage, incorporando ahora formularios, búsqueda en tiempo real, generación de folios, indicadores y reglas de negocio mediante un flujo de estados.

2. Situación problema

El departamento de TI de una universidad recibe solicitudes de soporte mediante mensajes, llamadas o de forma verbal. Esto dificulta saber qué problemas siguen pendientes, cuáles son prioritarios y cuáles ya fueron resueltos. Se requiere construir HelpDesk Lite, una aplicación web que centralice las incidencias y permita dar seguimiento a su ciclo de vida.

3. Restricciones tecnológicas

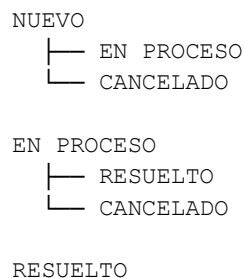
- Utilizar exclusivamente HTML5, CSS3 y JavaScript Vanilla.
- No utilizar Bootstrap, Tailwind CSS, React, Vue, Angular, jQuery ni frameworks similares.
- No utilizar backend, API REST ni base de datos.
- La persistencia deberá realizarse con localStorage.
- El proyecto deberá utilizar Git y un flujo GitFlow simplificado.
- No desarrollar directamente sobre la rama main.

4. Resultado esperado

Al finalizar, la aplicación deberá permitir:

- Crear tickets mediante un formulario.
- Generar un folio automático para cada ticket.
- Visualizar tickets mediante tarjetas.
- Buscar tickets por texto.
- Filtrar por estado y prioridad.
- Cambiar el estado respetando reglas de negocio.
- Cancelar tickets cuando corresponda.
- Mostrar indicadores calculados desde los datos.
- Conservar la información después de recargar el navegador.
- Funcionar correctamente en escritorio y móvil.

5. Flujo de estados obligatorio



└─ CERRADO

CERRADO → estado final

CANCELADO → estado final

No se permite cambiar libremente el estado. Por ejemplo, un ticket NUEVO no puede pasar directamente a CERRADO y un ticket CANCELADO no puede regresar a EN PROCESO.

6. Requerimientos funcionales

ID	Requerimiento	Prioridad
RF-01	Crear tickets de soporte.	Alta
RF-02	Generar automáticamente un folio único.	Alta
RF-03	Registrar título y descripción.	Alta
RF-04	Seleccionar categoría.	Alta
RF-05	Seleccionar prioridad.	Alta
RF-06	Registrar fecha y hora de creación.	Alta
RF-07	Mostrar todos los tickets registrados.	Alta
RF-08	Filtrar tickets por estado.	Alta
RF-09	Filtrar tickets por prioridad.	Alta
RF-10	Buscar tickets por texto en tiempo real.	Alta
RF-11	Cambiar el estado del ticket.	Alta
RF-12	Impedir transiciones de estado inválidas.	Alta
RF-13	Cancelar tickets cuando el estado lo permita.	Media
RF-14	Mostrar indicadores del dashboard.	Alta
RF-15	Persistir tickets en localStorage.	Alta
RF-16	Mantener interfaz responsive.	Alta
RF-17	Mostrar un estado vacío cuando no existan tickets.	Media
RF-18	Validar que el formulario no se envíe incompleto.	Alta

7. Datos mínimos del ticket

Cada ticket deberá contener como mínimo:

```
{
  id: 5,
  folio: 'HD-0005',
  title: 'Internet caído en laboratorio',
  description: 'No existe conexión en los equipos...',
  category: 'Red',
  priority: 'Crítica',
  status: 'Nuevo',
  createdAt: '2026-09-02T10:35:00'
}
```

8. Catálogos mínimos

Categorías:

- Hardware
- Software
- Red
- Accesos
- Correo
- Otro

Prioridades:

- Baja
- Media
- Alta
- Crítica

Estados:

- Nuevo
- En proceso
- Resuelto
- Cerrado
- Cancelado

9. Mockup de referencia - escritorio

HelpDesk Lite		+ Nuevo Ticket	
TOTAL 12	NUEVOS 4	PROCESO 5	RESUELTOS 3
Buscar [internet_____]			
Estado: [Todos] [Nuevo] [Proceso] [Resuelto] [Cerrado]			
Prioridad: [Todas ▼]			
HD-0005		CRÍTICA	
Internet caído en laboratorio			
Red · Creado: 02/09/2026 10:35			
Estado: EN PROCESO			
[Resolver] [Cancelar]			
HD-0004		ALTA	
Computadora no inicia			
Hardware · Creado: 02/09/2026 09:10			
Estado: NUEVO			
[Iniciar atención] [Cancelar]			

10. Mockup del formulario

NUEVO TICKET	
Título [_____]	
Descripción [_____ _____]	
Categoría	[Hardware ▼]
Prioridad	[Media ▼]
[Crear Ticket]	

11. Mockup móvil

HelpDesk Lite [+ Nuevo Ticket]	
TOTAL 12	NUEVOS 4
PROCESO 5	RESUELTOS 3
Buscar [_____]	
[Todos] [Nuevo] [Proceso]	
Prioridad [Todas ▼]	

HD-0005	CRÍTICA
Internet caído...	
Estado: EN PROCESO	
[Resolver] [Cancelar]	
HD-0004	ALTA
Computadora no inicia	
Estado: NUEVO	
[Iniciar] [Cancelar]	

El mockup define estructura y experiencia de uso. Los colores, tipografía y detalles visuales pueden ser propuestos por el alumno, siempre que exista buena legibilidad, jerarquía visual y comportamiento responsive.

12. Requerimientos HTML5

Se deberá demostrar el uso correcto de:

```
<header> <nav> <main> <section> <article> <footer>
<form> <label> <input> <textarea> <select> <option> <button>
```

- lang="es", charset UTF-8 y viewport.
- HTML semántico.
- Labels asociados correctamente con controles de formulario.
- CSS y JavaScript en archivos externos.
- Uso de id, class y atributos data-* cuando sea necesario.
- No colocar estilos inline.

13. Requerimientos CSS3

- Variables CSS mediante :root.
- Box Model.
- Flexbox.
- CSS Grid.
- Pseudo-clases como :hover y :disabled.
- Transiciones.
- Clases reutilizables.
- Media Queries.
- Diseño responsive.
- Diferenciación visual de prioridad y estado.
- Estado visual para botones deshabilitados.

14. Requerimientos JavaScript

- const y let.
- Arrays y objetos.
- Funciones y arrow functions.
- forEach(), filter(), find(), some(), includes(), map() cuando sea útil.
- querySelector() y querySelectorAll().

- `createElement()`, `classList` y `dataset`.
- `addEventListener()`.
- Evento `submit` y `preventDefault()`.
- Evento `input` para búsqueda en tiempo real.
- `trim()`, `toLowerCase()` e `includes()`.
- `Date` para fecha/hora.
- `padStart()` para el folio.
- `localStorage`, `JSON.stringify()` y `JSON.parse()`.

15. Reglas de negocio

ID	Regla
RN-01	El folio debe ser único y consecutivo con formato HD-0001.
RN-02	No se puede crear un ticket sin título, descripción, categoría y prioridad.
RN-03	Todo ticket nuevo inicia con estado Nuevo.
RN-04	Nuevo solo puede pasar a En proceso o Cancelado.
RN-05	En proceso solo puede pasar a Resuelto o Cancelado.
RN-06	Resuelto solo puede pasar a Cerrado.
RN-07	Cerrado y Cancelado son estados finales.
RN-08	Los indicadores deben calcularse desde el arreglo de tickets.
RN-09	La búsqueda debe actualizar resultados mientras el usuario escribe.
RN-10	Los filtros y búsqueda deben poder trabajar sobre los mismos datos.

16. Generación del folio

El formato requerido es:

```
HD-0001
HD-0002
HD-0003
...
HD-0099
HD-0100
```

Se deberá investigar y aplicar una solución basada en una secuencia y el método:

```
String(numero).padStart(4, '0')
```

No se deberá escribir manualmente el folio de cada ticket.

17. Dashboard dinámico

Los valores TOTAL, NUEVOS, EN PROCESO y RESUELTOS deberán recalcularse después de crear, cancelar o cambiar el estado de un ticket. No se permite escribir cantidades fijas en el HTML.

18. Búsqueda en tiempo real

El campo de búsqueda deberá reaccionar al evento input. Como mínimo deberá buscar coincidencias en el folio, título o descripción. La comparación deberá ser tolerante a mayúsculas/minúsculas.

19. Persistencia

Los tickets deberán guardarse en localStorage. Después de recargar la página, la aplicación deberá reconstruir la interfaz usando la información almacenada.

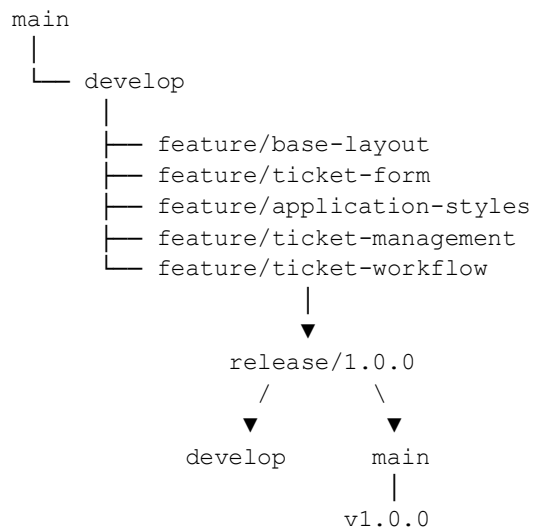
20. Estructura mínima del proyecto

```
helpdesk-lite/
├── css/
│   └── styles.css
├── js/
│   └── app.js
├── docs/
│   └── evidencias/
├── index.html
└── README.md
```

21. Desarrollo evolutivo por sesiones

Etap	Tema	Producto	Rama
Sesión 1	HTML5 + Git	Estructura semántica, formulario y dashboard base.	feature/base-layout y feature/ticket-form
Sesión 2	CSS3	Sistema visual, cards, prioridades, estados y responsive.	feature/application-styles
Sesión 3	JavaScript + DOM	Creación, folios, render, búsqueda y filtros.	feature/ticket-management
Sesión 4	Reglas + persistencia	Estados, dashboard, localStorage, cancelación y pruebas.	feature/ticket-workflow

22. GitFlow obligatorio



Cada feature deberá nacer desde develop y regresar mediante merge. La versión estable deberá llegar a main mediante release/1.0.0.

23. Commits esperados

```
git commit -m "feat: create semantic application structure"
git commit -m "feat: add ticket form"
```

```
git commit -m "style: create responsive ticket layout"
git commit -m "feat: create dynamic ticket rendering"
git commit -m "feat: generate sequential ticket folios"
git commit -m "feat: add real-time ticket search"
git commit -m "feat: filter tickets by status and priority"
git commit -m "feat: implement ticket state transitions"
git commit -m "feat: persist tickets in local storage"
git commit -m "fix: prevent invalid ticket transitions"
```

Evitar commits como: cambios, avance, prueba, final, final2 o ya quedó.

24. Evidencias obligatorias

- URL del repositorio.
- Captura de la aplicación en escritorio.
- Captura de la aplicación en móvil.
- Evidencia de creación de un ticket.
- Evidencia del folio automático.
- Evidencia de búsqueda en tiempo real.
- Evidencia de filtro por estado.
- Evidencia de filtro por prioridad.
- Evidencia Nuevo → En proceso.
- Evidencia En proceso → Resuelto.
- Evidencia Resuelto → Cerrado.
- Evidencia de una transición inválida impedida.
- Evidencia de persistencia después de recargar.
- Captura de Git Graph.
- Salida de git log --graph --oneline --decorate --all.
- Tag v1.0.0.

25. Evidencia del historial Git

```
git log --graph --oneline --decorate --all
```

La gráfica deberá permitir identificar claramente features, merges, develop, release, main y el tag final.

26. Criterios de evaluación

Criterio	Valor	Qué se evalúa
HTML5 y formulario	15%	Semántica, estructura, accesibilidad básica y formulario correcto.
CSS3 y responsive	15%	Layout, jerarquía visual, estados, prioridades y adaptación móvil.
JavaScript y DOM	25%	Render dinámico, eventos, búsqueda, filtros y manipulación correcta del DOM.
Reglas de negocio	15%	Transiciones válidas, validaciones y folio.
localStorage	10%	Persistencia y recuperación correcta.
Git/GitFlow	15%	Ramas, commits, merges, release, tag y evidencias.
Calidad general	5%	Orden, nombres, legibilidad y README.

27. Condiciones de entrega

- La aplicación debe abrir y funcionar sin errores en consola.
- No se aceptará una solución construida únicamente en un solo commit.
- No se aceptará desarrollo directo completo sobre main.
- Los datos deben generarse y renderizarse desde JavaScript donde corresponda.
- El historial Git forma parte de la calificación.
- La interfaz debe ser funcional tanto en escritorio como en móvil.
- El README debe explicar cómo ejecutar el proyecto y resumir sus funcionalidades.

28. Checklist final del alumno

- ☐ El formulario valida campos obligatorios.
- ☐ Se genera un folio HD-XXXX automáticamente.
- ☐ Los tickets aparecen sin recargar la página.
- ☐ Los filtros funcionan.
- ☐ La búsqueda funciona mientras escribo.
- ☐ Los indicadores se actualizan automáticamente.
- ☐ Las transiciones de estado válidas funcionan.
- ☐ Las transiciones inválidas están bloqueadas.
- ☐ Los datos permanecen después de recargar.
- ☐ La interfaz funciona en móvil.
- ☐ No existen errores en la consola.

- ☐ Git Graph muestra las ramas y merges.
- ☐ Existe release/1.0.0 en el historial.
- ☐ main contiene la versión final.
- ☐ Existe el tag v1.0.0.
- ☐ El repositorio contiene las evidencias solicitadas.

29. Reto adicional (opcional)

Para alumnos que terminen antes, implementar uno o más:

- Ordenar tickets por prioridad o fecha.
- Agregar contador de tickets críticos.
- Permitir editar título o descripción únicamente mientras el ticket esté Nuevo.
- Agregar modo oscuro usando CSS y localStorage.
- Agregar confirmación antes de cancelar.
- Registrar un pequeño historial de cambios de estado dentro del ticket.

30. Resultado de aprendizaje

Al concluir, el alumno deberá poder explicar cómo una aplicación frontend administra datos y estado sin backend, cómo el DOM representa esos datos visualmente, cómo localStorage permite persistencia local y cómo GitFlow conserva evidencia verificable de la evolución del software.